



Simulador de Prevención de Interbloqueo (deadlock)

Propósito

El propósito de este proyecto es desarrollar un simulador que implemente un algoritmo de prevención de interbloqueo (deadlock) en sistemas operativos. Los objetivos específicos son:

1. **Entender** y **explicar** el funcionamiento del algoritmo de prevención de interbloqueo.
2. **Diseñar** y **programar** un simulador que permita visualizar el proceso de prevención de interbloqueo.
3. **Probar** el simulador con diferentes escenarios para verificar su correcto funcionamiento.
4. **Documentar** el diseño y el uso del simulador.

El algoritmo se puede encontrar con más detalle en el libro de William Stallings a partir de la página 269 aproximadamente.

Consideraciones de implementación:

El formato de ejecución de su programa debe ser el siguiente:

./prevencion <nombre del archivo>

El archivo contiene un snapshot del flujo de recursos y ejecución de N cantidad de procesos, dicho archivo está definido por el siguiente formato:

N recursos

Vector recursos totales, este vector tiene el tamaño de N

Vector recursos disponibles, este vector tiene el tamaño de N

M procesos – Nombrar todos por P, los números van del [0,M-1]

Recursos asignados | Recursos necesarios para ejecución | Prioridad

Donde la prioridad va del 1 hasta el 10 siendo 1 mayor prioridad que 10.



Ejemplo:

3
6 5 10
3 3 2
5
1 0 0 | 4 4 4 | 1
0 1 2 | 1 4 6 | 3
0 0 3 | 3 2 5 | 2
1 0 3 | 4 4 8 | 6
1 1 0 | 5 1 10 | 10

NOTA: El input se define como vectores para hacer más sencillo el procesamiento y almacenamiento de datos del archivo. Sin embargo, la estructura de datos de almacenamiento que se va a manipular en la implementación del algoritmo de prevención de interbloqueo posee una estructura de tipo matriz, definida de la siguiente manera:

Recursos = $\mathbf{R} = (R_1, R_2, \dots, R_m)$		cantidad total de cada recurso en el sistema
Disponibles = $\mathbf{D} = (D_1, D_2, \dots, D_m)$		cantidad total de cada recurso no asignada a ningún proceso
Necesidad = $\mathbf{N} =$	$\begin{vmatrix} N_{11} & N_{12} & \dots & N_{1m} \\ N_{21} & N_{22} & \dots & N_{2m} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ N_{n1} & N_{n2} & \dots & N_{nm} \end{vmatrix}$	N_{ij} = necesidades del proceso i con respecto al recurso j
Asignación = $\mathbf{A} =$	$\begin{vmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{vmatrix}$	A_{ij} = asignación actual al proceso i con respecto al recurso j



Salida:

Si el algoritmo encuentra un estado inseguro y no puede continuar su ejecución, va a imprimir los procesos que faltan por finalizar, Ejemplo:

Es un estado inseguro

Procesos involucrados P1 P3 P4

Recursos disponibles 2 3 1

Para este algoritmo del banquero modificado, en este estado inseguro, se debe dar la opción al usuario de finalizar el algoritmo o terminar los procesos 1 a 1 hasta tener los recursos suficientes para continuar con el algoritmo, por cada proceso terminado continuar con el algoritmo y la asignación de recursos, hasta que todos los procesos finalicen, los procesos terminados a la fuerza deben ser clasificados como procesos terminados.

Implementar alguno de estos tres criterios para elegir un proceso a terminar y apropiarse de sus recursos:

- Aleatorio
- Prioridad, donde se termina el proceso con menor prioridad.
- Selectivo, donde 0 es el proceso 0, 1 es el proceso 1 2 es el proceso 2...

Si el algoritmo termina y encuentra un estado seguro, imprimir el posible orden de ejecución de los procesos y asignación de recursos. Por cada proceso debe imprimir su estado si finalizo su ejecución o fue terminado para apropiarse de sus recursos. Ejemplo:

Es un estado seguro

Estado de los procesos:

P0 Finalizado

P1 Finalizado

P2 Finalizado



P3 Finalizado

P4 Finalizado

Orden de asignación de recursos:

P2, P0, P1, P3, P4

Si el algoritmo tuvo que terminar procesos para finalizar su ejecución, tiene que indicarse qué procesos fueron terminados para continuar con la ejecución del algoritmo, en el orden de asignación de recursos en el que se tomó la decisión de terminarlos, ejemplo:

Hubo un estado inseguro

Estado de los procesos:

P0 Terminado

P1 Finalizado

P2 Finalizado

P3 Terminado

P4 Finalizado

Orden de asignación de recursos:

P2, P3 Terminado, P0 Terminado, P1, P4

Entregables

Su proyecto debe contener al menos los siguientes archivos al momento de ser entregado para revisión:

- Sus archivos de código fuente en C (archivos .c) y cabeceras relacionadas (archivos .h) necesarios para compilar el proyecto. Es obligatorio implementar al menos 1 cabecera.
- Un archivo Makefile que permita compilar el proyecto. El Makefile debe contener al menos las siguientes reglas:



1. all: debe permitir compilar el proyecto completo y producir el programa ejecutable correspondiente.
 2. clean: debe eliminar los archivos intermedios de compilación que se produzcan (por ejemplo, los archivos .o) y el ejecutable del proyecto.
- Un informe de mínimo 4 páginas en formato .PDF que contenga una descripción del programa desarrollado. Debe describir como está estructurado el código fuente e identificar los puntos de su código fuente donde se implementaron cada funcionalidad. Se debe anexar cada descripción/explicación con sus respectivas capturas. Además, se deben anexar casos de prueba, su estructura y su resultado en la ejecución de su solución.
 - Anexar los archivos de sus casos de prueba con resultados reales.

Consideraciones Generales:

- El proyecto deberá realizarse en grupo de 2 (dos) personas máximo.
- Para compilar el proyecto creado debe hacerse uso de un Makefile, el cual debe ser desarrollado por cada grupo, si el código es enviado sin el Makefile o el Makefile no funciona, el código no será evaluado.
- Los estudiantes son libres de organizar su código en diferentes cabeceras y secciones de código, es obligatorio el desarrollo y uso de cabeceras. De no cumplirse tendrá una penalización de 2 puntos.
- La implementación del proyecto será realizada utilizando el lenguaje de programación C, bajo un sistema Linux, de preferencia compatible con el sistema Ubuntu 24.04 LTS o derivados, debe ser compilable con cualquier versión actual de GCC. Ejemplo: gcc 13.3.0
- Usted deberá entregar el código fuente correspondiente al proyecto totalmente funcional y sin errores. Aquellos códigos que presenten errores de compilación no serán evaluados.
- No se aceptarán entregas fuera de la fecha y hora establecida.
- De acuerdo con lo establecido en la Ley de Universidades, las copias serán penalizadas con la nota mínima (0) para todos los involucrados.
- Se prohíbe el uso de IA generativas (LLM) para desarrollar el proyecto, de ser detectado la nota será severamente penalizada con la nota mínima (0).



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Sistemas Operativos (6004)
Proyecto #2



- El asunto del correo debe ser [SO]_[Proy2]_[Cedula1]_[Cedula2], si no cumple con el formato del asunto, la nota será severamente penalizada. De no cumplirse tendrá una penalización de 5 puntos.
- El código debe estar escrito de una manera clara, ordenada y bien documentada/comentada.
- Los entregables del proyecto deben ser enviados en un archivo .zip al correo laboratoriosisop@gmail.com.
- La fecha de entrega será determinada por el GDSO, se le avisará por el grupo de Telegram de la materia.

La distribución de puntos queda de la siguiente manera:

Informe - 4 pts

Defensa individual - 3 pts

Documentación/comentarios - 2 pts

Funcionalidad del código - 11 pts